

Contents

▢ Sliding Window — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	3
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	4
SECTION 6 — Time & Space Complexity	5
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	6
SECTION 9 — Real World Applications	6
SECTION 10 — Interview Strategy	7
SECTION 11 — Pattern Variations	7
SECTION 12 — Comparison With Other Patterns	8
SECTION 13 — Problem Classification	8
SECTION 14 — 30 Interview Problems	9
SECTION 15 — Common Mistakes	11
SECTION 16 — Optimization Techniques	12
SECTION 17 — Multiple Language Templates	12
SECTION 18 — Dry Runs	14
SECTION 19 — Advanced Concepts	14
SECTION 20 — Senior Engineer Insights	15
SECTION 21 — Cheat Sheet	15
SECTION 22 — Revision Notes (5-Minute Review)	15
SECTION 23 — Practice Roadmap	15
SECTION 24 — Memory Tricks	16
SECTION 25 — Final Summary	16

▢ Sliding Window — Complete Interview Handbook

Pattern #2 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Sliding Window section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Sliding Window maintains a **contiguous range** `[left, right]` over an array or string, expanding and contracting its boundaries as it scans once left-to-right, so that the range always represents “the current candidate subarray/substring” for the problem. Instead of recomputing a property (sum, count, distinct-characters) from scratch for every possible subrange, it **incrementally updates** that property as the window moves — turning an $O(n^2)$ or $O(n^3)$ brute force into $O(n)$ (or $O(n \cdot k)$ for bounded alphabets).

1.2 Why Was This Pattern Invented?

Brute-force “check every subarray” solutions recompute overlapping work: subarray `[2,5]` and `[2,6]` share almost all their elements, yet naive approaches recompute sums/counts from scratch for each. Sliding Window formalizes “reuse the work you already did for the previous window instead of restarting,” directly analogous to how dynamic programming reuses subproblem results.

1.3 Real Intuition Behind The Pattern

Picture a **rigid or elastic frame sliding across a filmstrip**. As it slides one frame to the right, you don’t re-examine the whole strip — you just add the new frame entering on the right and remove the one exiting on the left. The window’s “state” (sum, character counts, distinct count) is updated incrementally in $O(1)$ per step, not recomputed in $O(k)$.

1.4 Mental Model

Two pointers, `left` and `right`, define a **living, breathing range**. `right` always drives forward exploration (“try including more”); `left` only moves forward when the window becomes invalid or needs shrinking (“give back what’s not needed”). Both pointers move strictly forward — never backward — which is what bounds total work to $O(n)$.

1.5 Visual Explanation

```
String:  a  b  c  a  b  c  b  b
         L
         R           window="a", 1 distinct
         L R        window="ab", 2 distinct
         L   R      window="abc", 3 distinct
           L R      (shrunk: "bc") if constraint violated
```

Each step either grows the window (`right++`) or shrinks it (`left++`) — never restarts from scratch.

1.6 Simple Analogy

Sliding Window is like **reading a book through a moving bookmark-width viewport** — as you slide the viewport one word forward, you drop the leftmost word from your “currently reading” set and pick up the next word on the right, never re-reading everything from page one.

1.7 When Should I Immediately Think About Using This Pattern?

- The problem asks for the **longest/shortest/count of contiguous** subarray or substring satisfying some condition.
- Keywords like “at most K distinct,” “no repeating characters,” “sum equal to/at least/at most K ” over a **contiguous** range.
- You’d otherwise write a nested loop generating all subarrays ($O(n^2)$) and want $O(n)$.

SECTION 2 — Recognition Guide

2.1 Keywords In Problem Statement

Keyword	Signal
“contiguous subarray/substring”	Strong signal
“longest substring without repeating”	Classic variable window
“at most K distinct characters”	Variable window with a frequency map
“maximum sum subarray of size K”	Fixed-size window
“minimum window substring”	Variable shrinkable window
“average of subarrays of size K”	Fixed-size window

2.2 Hidden Hints

Constraint like $1 \leq s.length \leq 10^5$ with expected $O(n)$ rules out generating all substrings ($O(n^2)$). A problem that *sounds* like Two Pointers but explicitly involves a **size or sum threshold that changes as you scan** (not a static target pair) is Sliding Window, not Two Pointers.

2.3 Interview Clues

Interviewer mentions “contiguous,” or the brute force you propose is $O(n^2)/O(n^3)$ via nested loops over start/end indices — that’s your cue to compress into a window.

2.4 Common Trick Words

“At most” vs “exactly” (exactly-K is often solved as $\text{atMost}(K) - \text{atMost}(K-1)$, a well-known trick); “smallest window containing,” which implies a variable-shrink window with a “satisfied” condition tracked via counters.

2.5 What Interviewers Expect

Correct identification of **fixed vs variable window**, $O(1)$ incremental state updates (not $O(k)$ recomputation per step), and correct handling of the shrink condition (while-loop, not if).

2.6 When NOT To Use This Pattern

- Non-contiguous subsequences (window doesn’t apply — that’s usually DP or Two Pointers on separate indices).
- Condition isn’t **monotonic** with respect to window growth (e.g., “product is exactly K” oscillates non-monotonically and can break simple shrink logic — needs care or a different approach).
- Need every subarray’s value (not just optimal one) — window only tracks current state efficiently, not history, unless you explicitly log results.

SECTION 3 — Decision Framework

Is the target a CONTIGUOUS subarray/substring?

Yes

No → use Two Pointers / Hashing / DP instead

Is the window SIZE fixed (given K)?

Yes No

FIXED Does growing the window monotonically
WINDOW help/hurt the constraint (sum, distinct count)?

(slide

by 1) Yes → VARIABLE WINDOW (expand right, shrink left on violation)

No → condition non-monotonic → reconsider: Prefix Sum / Hashing / Brute force with pruning

Why: Fixed windows need no shrink logic — just add incoming, remove outgoing. Variable windows depend on monotonicity: growing the window must move the constraint predictably in one direction so shrinking from the left is always a valid corrective action, mirroring the Two Pointers exchange argument.

SECTION 4 — Why This Pattern Works

Mathematical: Let $f(\text{window})$ be the property being tracked (sum, distinct count). For fixed-size windows, f updates in $O(1)$: $f(w[i+1..i+k]) = f(w[i..i+k-1]) - a[i] + a[i+k]$. For variable windows, correctness relies on **monotonic invalidation**: once a window violates the constraint, shrinking from the left (removing the oldest, “least necessary” element) is guaranteed to move toward validity because every element removed can only relax the violated condition, never worsen it.

Logical: Each index enters the window exactly once (via `right++`) and leaves it exactly once (via `left++`), so total pointer movements are bounded by $2n = O(n)$.

Intuitive: Like a caterpillar inching forward — front end (right) explores, back end (left) catches up only when necessary — the total ground covered by each end is bounded by the strip’s length.

Correctness proof: Invariant — at every step, $[left, right)$ is the **longest/shortest valid window ending at right** found so far. Base case: empty window trivially valid. Inductive step: extending `right` either keeps the window valid (record it) or invalidates it, triggering `left` to advance until validity is restored — since invalidation is monotonic in window size (adding elements only ever adds to sum/distinct-count, never subtracts), advancing `left` is guaranteed to eventually restore validity or empty the window. Termination: `right` reaches n , all windows considered. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework

1. Initialize `left = 0`, window state (sum/frequency map/counter).
2. Loop `right` from 0 to $n-1$: add `arr[right]` to window state.
3. **Fixed window:** once `right - left + 1 == k`, record result, then remove `arr[left]` and `left++`.
4. **Variable window:** while window violates constraint, remove `arr[left]` from state and `left++`; then record result (max/min length, count, etc.) based on current valid window.
5. Return the tracked answer.

5.2 General Template — Variable Window

```
function slidingWindow(array, condition):  
    left = 0  
    windowState = {}  
    best = initialValue
```

```

for right in range(0, length(array)):
    add array[right] to windowState

    while windowState violates condition:
        remove array[left] from windowState
        left = left + 1

    best = update(best, right - left + 1)    # or other window-based metric

return best

```

5.3 Fixed-Size Window Template

```

function fixedWindow(array, k):
    windowSum = sum(array[0..k-1])
    best = windowSum

    for right in range(k, length(array)):
        windowSum = windowSum + array[right] - array[right - k]
        best = max(best, windowSum)

    return best

```

5.4 Interview Thinking Process

1. "This needs a contiguous subrange — I'll use Sliding Window."
2. "Is the window size fixed or does it grow/shrink based on a constraint?"
3. "I'll expand with `right`, and shrink with `left` only when the window becomes invalid — this keeps total work $O(n)$."
4. "I'll maintain window state incrementally ($O(1)$ per step), not recompute it."
5. "Let me dry-run a small example to confirm the shrink condition is correct before finalizing."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n)$	$O(k)$ or $O(\text{alphabet size})$ for frequency map	Each element added/removed from window at most once
Average Case	$O(n)$	$O(1)$ to $O(26)$ for character problems	Bounded alphabet keeps map size constant
Best Case	$O(n)$ (still must scan once)	$O(1)$	Even trivial cases require one full pass
Amortized	$O(n)$ despite nested while-loop for shrinking	$O(k)$	<code>left</code> advances at most n times total across the entire run, regardless of how many times the while-loop triggers per outer iteration

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Window larger than array	$k > n$	Return early / no valid window
Empty string/array	<code>"", []</code>	Return 0 or empty immediately
All identical elements	<code>"aaaa", k distinct=1</code>	Window never needs to shrink
Negative numbers in sum-based window	<code>[-1, 2, -3, 4]</code>	Fixed-size sum window still works ($O(1)$ update); variable “at least sum K ” breaks monotonicity — needs prefix sum + different approach instead
Single element equal to k	<code>[5], k=1</code>	Window trivially valid
No valid window exists	“min window substring” with impossible target	Return empty string / -1 sentinel
Duplicate characters requiring exact count	“permutation in string”	Use frequency map equality check, not just distinct count

Common mistakes: using `if` instead of `while` for shrink logic (can leave window invalid after only one shrink step); forgetting negative numbers break the “larger window = larger sum” monotonicity assumption used in “at least K sum” variants.

SECTION 8 — Pros & Cons

Advantages: $O(n)$ instead of $O(n^2)/O(n^3)$; $O(1)$ incremental updates; intuitive and highly reusable state (hashmaps/counters). **Disadvantages:** Only works for **contiguous** ranges; breaks down when the underlying condition isn’t monotonic with window size (e.g., negative numbers with sum thresholds); can be tricky to get shrink conditions exactly right. **Trade-offs:** Sliding Window ($O(n)$ time, $O(k)$ space) vs brute force ($O(n^2)$, $O(1)$ space) — always prefer Sliding Window when contiguity + monotonicity hold. **Limitations:** Not applicable to subsequence (non-contiguous) problems; struggles with “exact” conditions unless decomposed via the “atMost(K) – atMost($K-1$)” trick. **Inefficient when:** the alphabet/state space is unbounded and large, making the window-state map itself expensive to maintain (rare, but consider for exotic key spaces).

SECTION 9 — Real World Applications

Domain	Application
Google	Rate limiting (sliding window log/counter) for API quota enforcement
Amazon	AWS API Gateway / throttling — sliding window rate limiters count requests within a rolling time window
Meta	News Feed “trending” detection — rolling window counts of engagement events per time slice

Domain	Application
Netflix	Adaptive bitrate streaming — sliding window average of recent network throughput samples
Uber	Surge pricing computed over rolling time windows of ride requests per geo-cell
Banking/Payments	Fraud detection — flagging accounts with abnormal transaction counts within a rolling window
Networking	TCP congestion control (sliding window protocol) governs in-flight packet counts
Operating Systems	CPU scheduling — moving averages of recent load over a window of time slices
Databases	Rolling aggregate materialized views (e.g., “last 5 minutes” dashboards)
AI/ML	Time-series feature extraction — rolling mean/variance windows for model inputs

SECTION 10 — Interview Strategy

How seniors answer: “This is a contiguous-range problem, so I’ll maintain a window with incrementally updated state. Growing the window is monotonic in [sum/distinct count], so shrinking from the left when invalid is always safe and never skips a better answer.” They explicitly distinguish fixed vs. variable window aloud.

How juniors answer: They often write brute-force nested loops first without recognizing the incremental-update opportunity, or apply a sliding window template blindly without checking whether the underlying condition is actually monotonic (leading to silent bugs with negative numbers).

Typical follow-ups: “What if the array has negative numbers?” (sum-based shrink breaks — discuss prefix sum + monotonic deque alternative). “Can you find the count of all valid windows, not just the longest?” (discuss the atMost(K)-atMost(K-1) technique). “What if k changes dynamically per query?” (discuss precomputation trade-offs).

Optimization questions: “Can the frequency map be replaced with a fixed-size array?” (yes, for bounded alphabets — $O(1)$ space instead of $O(n)$).

SECTION 11 — Pattern Variations

Variation	Description	Example
Fixed-Size Window	Window size k never changes	Max Sum Subarray of Size K
Variable-Size (Longest)	Expand greedily, shrink on violation, track max length	Longest Substring Without Repeating Characters
Variable-Size (Shortest)	Expand until valid, then shrink to minimize	Minimum Window Substring, Minimum Size Subarray
Counting Windows	Count all valid windows, not just extremal one	Sum Subarrays with K Different Integers

Variation	Description	Example
Two-Pointer Hybrid with Monotonic Deque	Track window min/max efficiently	Sliding Window Maximum (Pattern #11)

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Two Pointers	Searches for a pair/triplet, not a size-optimized contiguous range	Static pair/triplet search on sorted data
Prefix Sum	Precomputes cumulative sums for $O(1)$ range-sum queries, no window state maintained incrementally in a scanning sense	Static range-sum queries, especially with negative numbers
Monotonic Queue/Deque	Tracks min/max within a window efficiently — often used <i>inside</i> a sliding window	Need running min/max, not just sum/count
Dynamic Programming	Handles non-contiguous or overlapping subproblem structures	Non-contiguous subsequence problems

Comparison Table

Aspect	Sliding Window	Two Pointers	Prefix Sum
Contiguity required	Yes	No (usually pair search)	Yes (range)
Handles negative numbers (sum)	Poorly (variable window)	N/A	Yes
Space	$O(k)$	$O(1)$	$O(n)$
Best for	Longest/shortest/count of valid contiguous range	Pair/triplet existence	Range sum/difference queries

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Fixed window, simple sum/average	Maximum Average Subarray I
Medium	Variable window with frequency map	Longest Substring Without Repeating Characters, Longest Repeating Character Replacement

Difficulty	Characteristics	Examples
Hard	Multiple constraints, exact-match frequency windows	Minimum Window Substring, Permutation in String
Very Hard	Combined with monotonic deque or counting decomposition	Sliding Window Maximum, Subarrays with K Different Integers

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Sliding Window Applies	Learning Objective
1	Maximum Sum Subarray of Size K	Easy	Amazon, Microsoft	Fixed window $O(1)$ update	Foundational fixed window
2	Maximum Average Subarray I	Easy	Amazon	Fixed window	Fixed window with average
3	Longest Substring Without Repeating Characters	Medium	Amazon, Meta, Google, Microsoft	Variable window with set/map	Core variable window
4	Longest Repeating Character Replacement	Medium	Google, Meta	Variable window with max-frequency tracking	Constraint-based shrink
5	Minimum Size Subarray Sum	Medium	Meta, Amazon	Variable shrinking window (sum $\geq$ target)	Shortest valid window
6	Minimum Window Substring	Hard	Meta, Amazon, Microsoft, Uber	Variable window with exact frequency match	Advanced shrink logic
7	Permutation in String	Medium	Amazon, Microsoft	Fixed window with frequency map equality	Frequency-match window
8	Find All Anagrams in a String	Medium	Meta, Amazon	Fixed window frequency counting	Counting valid windows
9	Subarrays with K Different Integers	Hard	Google, Uber	atMost(K) - atMost(K-1) decomposition	Advanced counting trick

#	Problem	Difficulty	Companies	Why Sliding Window Applies	Learning Objective
10	Fruit Into Baskets	Medium	Amazon, Google	At-most-2-distinct variable window	Distinct-count constraint
11	Longest Substring with At Most K Distinct Characters	Medium	Google, Meta	Variable window with distinct-count map	Distinct-count constraint
12	Sliding Window Maximum	Hard	Amazon, Google, Meta	Window + monotonic deque hybrid	Cross-pattern combination
13	Longest Subarray of 1's After Deleting One Element	Medium	Google	Variable window with defect budget	Budgeted constraint window
14	Max Consecutive Ones III	Medium	Google, Amazon	Variable window with flip budget	Budgeted constraint window
15	Grumpy Bookstore Owner	Medium	Amazon	Fixed window maximizing gain	Fixed window with gain calculation
16	Count Number of Nice Subarrays	Medium	Google	atMost(K) decomposition on odd counts	Transform constraint into countable form
17	Binary Subarrays With Sum	Medium	Google, Amazon	atMost(K) decomposition	Counting exact-sum windows
18	Subarray Product Less Than K	Medium	Amazon, Meta	Variable window with product constraint	Non-sum constraint window
19	Longest Substring with At Least K Repeating Characters	Medium	Google	Divide-based sliding window variant	Recursive window partitioning
20	Replace the Substring for Balanced String	Medium	Google	Variable window minimizing replacement	Complement-window technique
21	Frequency of the Most Frequent Element	Medium	Google, Amazon	Sorted array + variable window with budget	Sorting + window combination

#	Problem	Difficulty	Companies	Why Sliding Window Applies	Learning Objective
22	Minimum Number of K Consecutive Bit Flips	Hard	Google	Window + difference array hybrid	Advanced window state tracking
23	Max Sum of Rectangle No Larger Than K	Hard	Google	2D extension of window/prefix concepts	Dimensional extension
24	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	Hard	Google, Meta	Window + monotonic deque for min/max	Combined pattern mastery
25	Count Subarrays With Fixed Bounds	Hard	Google	Multi-condition variable window	Multiple simultaneous constraints
26	Number of Substrings Containing All Three Characters	Medium	Amazon	Variable window with 3-character frequency tracking	Multi-character constraint
27	Maximum Erasure Value	Medium	Amazon	Variable window with uniqueness constraint	Sum + uniqueness combination
28	Shortest Subarray with Sum at Least K	Hard	Google, Uber	Window fails due to negatives — monotonic deque + prefix sum needed	Understanding pattern limits
29	K Radius Subarray Averages	Medium	Amazon	Fixed window average	Fixed window practice
30	Continuous Subarray Sum	Medium	Meta, Amazon	Prefix sum + modulo hybrid (not pure window)	Recognizing pattern boundaries

SECTION 15 — Common Mistakes

1. Using `if` instead of `while` for shrinking — leaves invalid windows uncorrected. *Fix*: always `while` on the violation condition.
2. Recomputing window state from scratch each step ($O(k)$ per step $\rightarrow O(nk)$ total) instead of incrementally updating. *Fix*: always add/remove only the entering/exiting element.

3. Forgetting that negative numbers break sum-monotonicity for variable windows. *Fix*: recognize when Prefix Sum + Monotonic Deque is the correct alternative.
4. Off-by-one on window length ($\text{right} - \text{left} + 1$ vs $\text{right} - \text{left}$). *Fix*: always dry-run with a 2-3 element window first.
5. Not resetting/cleaning window state correctly when shrinking removes the last occurrence of a character. *Fix*: delete the key from the map when count hits 0, don't leave stale zero-count entries if the logic depends on map size.

Why people fail: they pattern-match the “two pointers scanning forward” shape without verifying the monotonicity assumption holds for their specific condition — this passes on easy problems but silently fails on medium/hard variants with subtler constraints (negative numbers, exact-match requirements).

SECTION 16 — Optimization Techniques

- **Time:** Replace hashmap frequency counters with fixed-size arrays (e.g., size 26 for lowercase letters) for $O(1)$ real time instead of amortized hashmap overhead.
 - **Space:** Track only a “distinct count” integer instead of enumerating the full frequency map when only distinctness matters, not exact counts.
 - **Readability:** Name your window boundary variables `windowStart/windowEnd` explicitly in production-quality interview code; encapsulate the shrink condition in a clearly named boolean.
 - **Interview performance:** State the monotonicity assumption explicitly before coding; this preempts the most common follow-up question.
-

SECTION 17 — Multiple Language Templates

Java

```
public int lengthOfLongestSubstring(String s) {
    int[] lastSeen = new int[128];
    Arrays.fill(lastSeen, -1);
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastSeen[c] >= left) left = lastSeen[c] + 1;
        lastSeen[c] = right;
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

JavaScript

```
function lengthOfLongestSubstring(s) {
    const lastSeen = new Map();
    let left = 0, best = 0;
    for (let right = 0; right < s.length; right++) {
        const c = s[right];
        if (lastSeen.has(c) && lastSeen.get(c) >= left) left = lastSeen.get(c) + 1;
        lastSeen.set(c, right);
        best = Math.max(best, right - left + 1);
    }
}
```

```

    }
    return best;
}

```

PHP

```

function lengthOfLongestSubstring(string $s): int {
    $lastSeen = [];
    $left = 0; $best = 0;
    for ($right = 0; $right < strlen($s); $right++) {
        $c = $s[$right];
        if (isset($lastSeen[$c]) && $lastSeen[$c] >= $left) $left = $lastSeen[$c] + 1;
        $lastSeen[$c] = $right;
        $best = max($best, $right - $left + 1);
    }
    return $best;
}

```

Python

```

def length_of_longest_substring(s):
    last_seen = {}
    left = 0
    best = 0
    for right, c in enumerate(s):
        if c in last_seen and last_seen[c] >= left:
            left = last_seen[c] + 1
        last_seen[c] = right
        best = max(best, right - left + 1)
    return best

```

Go

```

func lengthOfLongestSubstring(s string) int {
    lastSeen := make(map[byte]int)
    left, best := 0, 0
    for right := 0; right < len(s); right++ {
        c := s[right]
        if idx, ok := lastSeen[c]; ok && idx >= left {
            left = idx + 1
        }
        lastSeen[c] = right
        if right-left+1 > best {
            best = right - left + 1
        }
    }
    return best
}

```

C++

```

int lengthOfLongestSubstring(string s) {
    vector<int> lastSeen(256, -1);

```

```

int left = 0, best = 0;
for (int right = 0; right < (int)s.size(); right++) {
    char c = s[right];
    if (lastSeen[c] >= left) left = lastSeen[c] + 1;
    lastSeen[c] = right;
    best = max(best, right - left + 1);
}
return best;
}

```

SECTION 18 — Dry Runs

Small Input

`s = "abcabcbb"`

```

right=0(a): window="a", best=1
right=1(b): window="ab", best=2
right=2(c): window="abc", best=3
right=3(a): 'a' seen at 0 >= left(0) → left=1, window="bca", best=3
right=4(b): 'b' seen at 1 >= left(1) → left=2, window="cab", best=3
right=5(c): 'c' seen at 2 >= left(2) → left=3, window="abc", best=3
right=6(b): 'b' seen at 4 >= left(3) → left=5, window="cb", best=3
right=7(b): 'b' seen at 6 >= left(5) → left=7, window="b", best=3
Result: 3

```

Large Input (Conceptual)

For a 100,000-character string, `left` and `right` each advance at most 100,000 times total — the algorithm remains a single $O(n)$ pass regardless of how many times the shrink condition triggers, since total shrink-steps are bounded by total elements ever added.

Corner Case

`s = ""` → loop never executes, return 0 immediately. `s = "bbbbbb"` → every character is a repeat; window shrinks to size 1 repeatedly; best stays 1.

SECTION 19 — Advanced Concepts

- **atMost(K) – atMost(K–1) trick:** converts “exactly K distinct” counting problems into two “at most K” sliding window subroutines — a powerful, broadly reusable transformation.
- **Sliding Window + Monotonic Deque:** for problems needing the window’s running min/max (not just sum/count), maintain a deque of candidate indices in monotonic order alongside the window (see Pattern #11).
- **Divide-based sliding window:** for “at least K repeating characters,” split the string at characters violating the global frequency constraint and recurse — a hybrid of window and divide-and-conquer.
- **Mathematical observation:** window length changes are always ± 1 per step for `right/left`, so any quantity that changes *linearly* with window membership (sum, count) can be updated in $O(1)$; quantities that don’t (e.g., median, product with zero) need special handling.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Sliding Window as a specific instance of the broader principle of **incremental recomputation** — the same idea behind caching, streaming aggregations, and reactive programming. They evaluate candidates not on whether they can produce the “two-pointer scan” shape, but on whether they can identify *which state must be tracked incrementally* and *why the shrink condition is monotonic*. Common follow-ups probe generalization: “How would you compute this over a live data stream where you can’t look back?” (leads to online/streaming algorithm discussion) or “How would this scale if k varied per query across millions of queries?” (leads to precomputation/segment tree discussion).

SECTION 21 — Cheat Sheet

PATTERN: Sliding Window

RECOGNIZE: contiguous subarray/substring + longest/shortest/count with a constraint

TEMPLATE:

```
left = 0; state = {}
for right in range(n):
    add arr[right] to state
    while state violates constraint:
        remove arr[left] from state; left++
    update answer using [left, right]
```

COMPLEXITY: $O(n)$ time, $O(k)$ space (k = distinct states / alphabet size)

KEY PROOF: window growth is monotonic in the tracked property → shrink from left is always safe

WATCH FOR: negative numbers (breaks sum monotonicity), while vs if for shrink, exact vs at-most conditions

DOESN'T APPLY WHEN: non-contiguous subsequences, non-monotonic constraints

SECTION 22 — Revision Notes (5-Minute Review)

- Sliding Window = incremental $O(1)$ state updates over a contiguous, forward-moving range.
 - Fixed window: add incoming, remove outgoing, no shrink loop.
 - Variable window: expand with right, shrink with while on violation.
 - `atMost(K)` – `atMost(K-1)` converts “exactly K ” into two window subroutines.
 - Negative numbers break naive sum-based shrink logic — recognize and pivot to Prefix Sum/Deque.
 - Combine with Monotonic Deque when you need running min/max inside the window.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Fixed window mechanics	Maximum Sum Subarray of Size K , Maximum Average Subarray I (643)
Intermediate	Variable window with maps	Longest Substring Without Repeating Characters (3), Fruit Into Baskets (904)

Stage	Focus	Recommended LeetCode Problems
Advanced	Exact-match / budgeted constraints	Minimum Window Substring (76), Max Consecutive Ones III (1004), Permutation in String (567)
Expert	Hybrid patterns / counting decomposition	Subarrays with K Different Integers (992), Sliding Window Maximum (239), Shortest Subarray with Sum at Least K (862)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Grow, Check, Shrink” (GCS) — Grow with right, Check constraint, Shrink with left.
 - **Visualization:** A trombone slide again, but this time think of an **inchworm** — front end stretches forward, back end catches up only when it must.
 - **Recognition shortcut:** See “contiguous” + “longest/shortest/count” → window. See “pair/triplet” + “sorted” → Two Pointers instead.
-

SECTION 25 — Final Summary

Sliding Window converts brute-force re-scanning of every subarray/substring into a single $O(n)$ pass by maintaining **incrementally updated window state** and exploiting the **monotonicity** of that state with respect to window growth. The one rule to remember forever: **grow greedily with right, shrink defensively with left only when invalid, and never recompute what you can incrementally update**. When that monotonicity assumption breaks (typically due to negative numbers or non-linear aggregate functions), it’s your signal to pivot to Prefix Sum, Monotonic Deque, or a specialized hybrid instead of forcing the window template to fit.